ENTERPRISE ARCHITECTURE REFERENCE & CODEBASE HANDBOOK

SettleFlow: Comprehensive Technical Manual & Line-by-Line Codebase Blueprint

Full-Stack Payment Orchestration, Resilience4j Circuit-Aware Smart Routing, Multi-PSP Mock Microservices, Python Settlement Reconciliation Engine & Cloud-Native Kubernetes Infrastructure

Executive Purpose: This document is the exhaustive technical specification and line-by-line audit of the **SettleFlow** payment platform. It covers every layer: Spring Boot 3.3 Java backend, Next.js 14 React frontend, Node.js mock PSP microservices, standalone Python settlement reconciliation engine, Docker multi-stage containerization, Kubernetes manifests with Kustomize, Cloudflare Tunnel ingress, and GitHub Actions CI/CD workflows. Each file, class, method, REST endpoint, and infrastructure configuration is analyzed in full detail with functional rationale, runtime mechanics, fault-tolerance behavior, and future scalability blueprints.

Platform Version: 1.0.0-PROD	**Target Environment:** Local / Hybrid / Kubernetes
Backend Engine: Java 17 LTS / Spring Boot 3.3.2	**Frontend Stack:** Next.js 14 / TypeScript / TailwindCSS
Resilience Framework: Resilience4j Circuit Breakers	**Database:** PostgreSQL 16 & H2 In-Memory Fallback
Reconciliation: Python 3.12 Standalone Worker	**Cluster Ingress:** Nginx Ingress & Cloudflare Zero-Trust Tunnel

1. Complete Technology Stack & Tools Deep-Dive

SettleFlow employs an enterprise-grade multi-tier architecture engineered for ultra-high availability, strict financial consistency, and automated self-healing. The table below details every core tool and library integrated into the platform, why it was chosen, how it works internally, and how it contributes to system stability.

Tool / Technology	Role in SettleFlow	Technical Mechanics & Production Benefit
Java 17 LTS	Backend Core Runtime	Provides modern language features (Java Records, sealed interfaces, pattern matching for switch), enhanced garbage collection (ZGC/G1), strong memory safety, and high-throughput thread execution.
Spring Boot 3.3.2	Transaction Orchestrator & REST Framework	Provides IoC dependency injection, Spring Data JPA, declarative validation (Hibernate Validator), embedded Tomcat web container, and synchronous non-blocking RestClient.
Resilience4j 2.2.0	Fault Tolerance & Circuit Breaking	Monitors outbound PSP call failure rates across a sliding window of calls. Automatically transitions degraded PSPs from CLOSED to OPEN, fast-failing traffic to prevent cascade bottlenecks.
PostgreSQL 16	Primary Relational Database	ACID transactional persistence for ledger entries. Ensures strict numeric precision (NUMERIC(19,2)), indexed idempotency keys, and concurrent transaction safety.
H2 Database Engine	In-Memory Local Fallback	Zero-dependency local database enabling instant developer bootup and rapid integration testing without spinning up PostgreSQL containers.
Next.js 14 (App Router)	Frontend Web Architecture	React Server Components, client-side hydration, file-based routing (/, /checkout, /dashboard), optimized bundle splitting, and robust build-time type verification.
TypeScript 5 & React 18	Frontend UI Logic & Type Safety	Eliminates runtime JavaScript type errors. Manages reactive component state (useState, useEffect, useMemo) and synchronous UI updates.
TailwindCSS & Vanilla CSS Tokens	Design System & Styling	Semantic CSS variables (--surface-1, --accent, --bg-success) for responsive layout control, light/dark adaptability, and polished micro-animations.
Node.js (v20)	Mock PSP Microservices	Zero-dependency standalone HTTP servers running Alpha (8081), Beta (8082), and Gamma (8083). Simulates real-world latency jitter, configurable failure rates, and settlement ledger exports.
Python 3.12 (urllib/json)	Settlement Reconciliation Worker	Batch reconciliation auditor that cross-references internal transactions with external PSP settlement reports to detect missing transactions, amount discrepancies, and status mismatches.
Docker & Multi-Stage Builds	Containerization & Security	Produces minimal Alpine-based runtime containers with non-root security users (settleflow:settleflow), reducing attack surface and container image footprint.
Kubernetes & Kustomize	Container Orchestration	Declarative cluster management: Deployments, Services, ConfigMaps, Ingress controllers, and automated scheduled CronJobs (0 6 * * *).
Cloudflare Tunnel	Zero-Trust Public Ingress	Exposes the local Kubernetes cluster to the public Internet securely over an encrypted outbound tunnel without exposing open inbound router ports.
GitHub Actions	Automated CI/CD Pipeline	Continuous integration workflows testing Maven builds, Next.js typechecks, mock PSP health curls, and Python reconciliation verification on every commit.

2. System Architecture & Core Execution Flows

SettleFlow coordinates the entire payment lifecycle across multiple acquirers with guaranteed idempotency, dynamic circuit-aware failover, and automated reconciliation. Below is an architectural overview of how requests flow through the platform.

2.1 Transaction Finite State Machine (FSM)

Every transaction strictly adheres to a deterministic finite state machine defined in TransactionState.java. Illegal transitions throw an immediate IllegalStateException, guaranteeing that no transaction can jump into an invalid financial state.

From State	Allowed Next State(s)	Trigger / Condition
PENDING	ROUTING	Transaction saved in database; state machine initiated.
ROUTING	PROCESSING	PspRouter selects healthy PSP based on rule match & circuit state.
PROCESSING	SETTLED	PSP returns HTTP 200 / approved response.
PROCESSING	RETRYING	PSP fails or times out, but attempt count < MAX_ATTEMPTS (3).
PROCESSING	FAILED	PSP fails and attempt count reaches MAX_ATTEMPTS (3). Terminal.
RETRYING	ROUTING	Re-enters router to select secondary/fallback healthy provider.
SETTLED	REFUNDED	Admin/customer support triggers refund on settled transaction.
FAILED / REFUNDED	*None (Terminal)*	Terminal states. No further mutations permitted.

2.2 Resilience4j Circuit Breaker Mathematical Model

Each payment provider is encapsulated within a Resilience4j Circuit Breaker configured in ResilienceConfig.java:
• **Sliding Window Size:** 10 calls (monitors the last 10 outcomes).
• **Failure Rate Threshold:** 50.0% (if ≥ 50% of the last 10 calls fail, the circuit flips to OPEN).
• **Minimum Calls:** 4 calls (circuit requires at least 4 calls before computing failure rate).
• **Wait Duration in OPEN State:** 15 seconds (cooldown window where all traffic is fast-failed immediately).
• **Permitted Calls in HALF_OPEN:** 3 probe calls to test if the provider has recovered.

2.3 Smart Routing & Fallback Algorithm

When PspRouter.selectFor(tx) executes, it evaluates providers in three hierarchical tiers:
1. **Rule Matching:** Queries active rules matching currency, min amount, and max amount. If matched, it checks if the preferredPsp circuit is CLOSED/HALF_OPEN. If healthy, it routes to preferred.
2. **Dynamic Fallback:** If preferred is degraded (OPEN), it checks the fallbackPsp. If fallback is healthy, it routes to fallback.
3. **Healthy Pool Round-Robin:** If no rule matches or both preferred/fallback are degraded, it filters all registered PSPs whose circuits are not OPEN and round-robins across them.
4. **Degraded Probe Fallback:** If all circuits are OPEN, it round-robins to allow half-open probe calls to test recovery.

3. Exhaustive File-by-File & Line-by-Line Codebase Audit

This section documents every file in the SettleFlow repository. Every module, class, method, and configuration parameter is analyzed line-by-line / block-by-block with technical rationale, runtime mechanics, and error handling notes.

3.1 Root Project Files

File: docker-compose.yml — Multi-Container Local Orchestration Stack

- **Lines 1-26 (PostgreSQL Service):** Configures postgres:16-alpine with container name settleflow-postgres, db credentials (settleflow/password123), persistent volume pgdata, and healthcheck pg_isready running every 5s with 5 retries.
- **Lines 28-42 (PSP Mocks Service):** Builds ./psp-mocks exposing ports 8081 (Alpha), 8082 (Beta), and 8083 (Gamma) on the settleflow-net bridge network.
- **Lines 44-71 (Spring Boot Backend Service):** Builds ./backend, waits for postgres service_healthy and psp-mocks service_started. Injects Spring DataSource connection string, JPA PostgreSQL dialect, CORS origin (http://localhost:3000), and internal PSP URLs.
- **Lines 73-89 (Next.js Frontend Service):** Builds ./frontend, exposes port 3000:3000, injects NEXT_PUBLIC_BACKEND_URL=http://localhost:8080.
- **Lines 91-105 (Reconciliation Worker Service):** Builds ./reconciliation, injects BACKEND_URL=http://backend:8080, and runs standalone settlement ledger cross-checks.

File: deploy.ps1 / deploy.sh — Automated Deployment Automation Scripts

- **Parameters & Flags:** Supports -BuildOnly (build container images only), -DeployOnly (skip build, apply manifests), and -Compose (deploy via docker compose up -d).
- **Docker Build Steps:** Builds ghcr.io/moni-sm/settleflow-backend, frontend, psp-mocks, and recon worker images.
- **Kubernetes Apply & Rollout:** Executes kubectl apply -k k8s/ and waits with kubectl rollout status on postgres, psp-mocks, backend, and frontend deployments with 120s-180s timeouts.

File: NOTES.md & README.md — Project Architecture Notes & Runbooks

- **Content & Scope:** Documents port allocations (3000 frontend, 8080 backend, 8081 Alpha, 8082 Beta, 8083 Gamma), manual execution commands without Docker, seed credentials (ops@settleflow.dev / demo1234), and progress logs.

3.2 Backend Core & Configuration Layer

File: backend/pom.xml — Maven Project Object Model & Dependency Graph

- **Lines 1-23 (Maven Coordinates & Java 17):** Declares parent spring-boot-starter-parent v3.3.2, artifact com.settleflow:backend:0.1.0, and Java version 17.
- **Lines 24-37 (Spring Boot Starters):** Includes spring-boot-starter-web (MVC, REST, embedded Tomcat), spring-boot-starter-data-jpa (Hibernate 6, Spring Data Repositories), and spring-boot-starter-validation (Jakarta Bean Validation).
- **Lines 38-47 (Database Drivers):** Includes org.postgresql:postgresql for production and com.h2database:h2 runtime dependency for zero-config local testing.
- **Lines 48-55 (Resilience4j Starter):** Includes io.github.resilience4j:resilience4j-spring-boot3 v2.2.0 for circuit breaking, sliding window metrics, and fallback execution.
- **Lines 63-71 (Build Plugins):** Configures spring-boot-maven-plugin for repackaging executable uber-JARs.

File: backend/src/main/resources/application.yml — Spring Boot Environment Configuration

- **Lines 1-3 (Server Config):** Configures server.port: 8080 for HTTP API listeners.
- **Lines 4-8 (H2 Console):** Enables H2 web console at /h2-console for runtime in-memory database inspection.
- **Lines 9-13 (Datasource Fallback):** Configures SPRING_DATASOURCE_URL with default fallback to in-memory H2 jdbc:h2:mem:settleflow;DB_CLOSE_DELAY=-1.
- **Lines 14-22 (JPA & Hibernate):** Sets ddl-auto: update, show-sql: true, format_sql: true for clear query observability.

- **Lines 23-25 (CORS Policy):** Sets cors.allowed-origin to http://localhost:3000 to enable secure browser fetch calls from the Next.js frontend.

File: backend/src/main/resources/schema.sql — Relational Database Schema DDL

- **Lines 4-16 (transactions table):** Defines id UUID PRIMARY KEY, player_id VARCHAR(255) NOT NULL, amount NUMERIC(19,2) NOT NULL, currency VARCHAR(10) NOT NULL, type VARCHAR(32) NOT NULL, psp VARCHAR(64), status VARCHAR(32) NOT NULL, attempt_count INT DEFAULT 0, idempotency_key VARCHAR(255) UNIQUE, created_at TIMESTAMPTZ, updated_at TIMESTAMPTZ.
- **Lines 18-20 (Performance Indexes):** Creates indexes on player_id, status, and idempotency_key for sub-millisecond query lookups.

File: SettleFlowApplication.java — Spring Boot Bootstrap Entrypoint

- **Lines 6-11:** Annotated with @SpringBootApplication; runs SpringApplication.run(SettleFlowApplication.class, args) to initialize the Spring IoC container, auto-scan components, and launch embedded Tomcat.

File: config/ResilienceConfig.java — Resilience4j Circuit Breaker Configuration & Bean Wiring

- **Lines 16-28 (circuitBreakerRegistry Bean):** Defines custom CircuitBreakerConfig with 50% failure rate threshold, 10-call sliding window, 4 minimum calls, 15s wait duration in OPEN, and 3 permitted calls in HALF_OPEN.
- **Lines 30-38 (Property Injection):** Injects ${psp.alpha.url}, ${psp.beta.url}, and ${psp.gamma.url} with localhost defaults.
- **Lines 39-55 (PspClient Beans):** Registers pspAlphaClient, pspBetaClient, and pspGammaClient as HttpPspClient instances wrapped with dedicated circuit breakers.

File: config/DataInitializer.java — Idempotent Database Seeding Engine

- **Lines 25-51 (CommandLineRunner Component):** Injects repositories for rules, transactions, players, recon runs, discrepancies, and audit logs.
- **Lines 53-61 (Routing Rules Seed):** Seeds default EUR High-Priority (psp-alpha -> psp-gamma), GBP Priority (psp-gamma -> psp-alpha), USD Global (psp-alpha -> psp-beta), and VIP High-Roller rules.
- **Lines 63-72 (Player KYC Seed):** Seeds 5 player profiles with verified KYC tiers, risk scores, daily deposit limits, and spend histories.
- **Lines 74-101 (Transactions Seed):** Seeds settled baseline transactions (TXN-88213, TXN-88212, TXN-88211, TXN-88210).
- **Lines 103-115 (Recon Runs & Discrepancies Seed):** Seeds historical reconciliation batches and open discrepancies (MISSING_IN_PSP, AMOUNT_MISMATCH, STATUS_MISMATCH).
- **Lines 117-126 (Audit Logs Seed):** Seeds immutable compliance audit records for circuit overrides, rule mutations, and KYC changes.

3.3 Backend Transaction Domain & State Machine

File: transaction/Transaction.java — Core Financial Transaction JPA Entity

- **Lines 8-59 (Fields & JPA Annotations):** Declares @Id @GeneratedValue UUID id, reference string, playerId, playerName, amount (BigDecimal), currency, @Enumerated TransactionType, method, psp, @Enumerated TransactionState, attemptCount, riskScore, refundReason, refundedAmount, @Column(unique=true) idempotencyKey, createdAt, and updatedAt.
- **Lines 64-80 (Constructors):** Initializes pending transaction, sets status=PENDING, generates timestamp, and assigns default reference TXN-XXXXX.
- **Lines 84-135 (Getters/Setters):** Provides full accessors and mutators; setStatus updates updatedAt to Instant.now() automatically; incrementAttemptCount increments attemptCount by 1.

File: transaction/TransactionState.java — Transaction State Machine Enumeration

- **Lines 3-11:** Defines PENDING, ROUTING, PROCESSING, SETTLED, RETRYING, FAILED, and REFUNDED states.

File: transaction/TransactionType.java — Transaction Operation Type

- **Lines 3-7:** Defines DEPOSIT, WITHDRAWAL, and REFUND financial transaction types.

File: transaction/CreateTransactionRequest.java — Validated Inbound Request Record

- **Lines 9-17:** Java record with @NotBlank playerId, playerName, @NotNull @Positive BigDecimal amount, @NotBlank currency, @NotNull TransactionType type, method, and idempotencyKey.

File: transaction/TransactionResponse.java — Outbound Projection DTO Record

- **Lines 8-48:** Read-only record encapsulating transaction state. Includes static factory method TransactionResponse.from(Transaction tx) for clean entity-to-DTO mapping.

File: transaction/TransactionRepository.java — Spring Data JPA Repository

- **Lines 8-10:** Extends JpaRepository; declares Optional findByIdempotencyKey(String idempotencyKey) for deduplication queries.

File: transaction/TransactionService.java — Transaction Lifecycle & State Machine Orchestrator

- **Lines 15-23 (Dependencies & Constants):** Defines MAX_ATTEMPTS = 3; injects TransactionRepository and PspRouter.
- **Lines 29-42 (create Method):** Checks idempotencyKey in repository. If present, returns existing transaction immediately (safe replay). Otherwise creates new transaction, persists to DB, runs drive(tx), and saves final state.
- **Lines 52-64 (refund Method):** Validates that transaction is in SETTLED state (throws IllegalStateException otherwise), sets status to REFUNDED, records refund reason and refunded amount, and persists changes.
- **Lines 69-93 (drive State Loop):** Transitions PENDING -> ROUTING. Loop: selects PSP via pspRouter.selectFor(tx), transitions ROUTING -> PROCESSING, increments attemptCount, executes psp.attempt(tx). If true -> transitions to SETTLED and returns. If attemptCount >= 3 -> transitions to FAILED and returns. Otherwise transitions PROCESSING -> RETRYING -> ROUTING and loops to try next provider.
- **Lines 95-117 (transition & isLegalTransition):** Enforces legal state transitions according to FSM graph. Throws IllegalStateException on invalid state jumps.

File: transaction/TransactionController.java — Transaction REST API Endpoints

- **Lines 25-36 (POST /api/transactions):** Validates request body with @Valid; calls service.create(); returns HTTP 201 CREATED with TransactionResponse.
- **Lines 38-43 (GET /api/transactions/{id}):** Looks up transaction by UUID; returns 200 OK or 404 NOT FOUND.
- **Lines 45-50 (GET /api/transactions):** Returns list of all transactions projected to TransactionResponse.
- **Lines 52-63 (POST /api/transactions/{id}/refund):** Accepts optional amount and reason; calls service.refund(id, amount, reason) and returns updated transaction.

3.4 Backend Routing & PSP Resilience Layer

File: routing/RoutingRule.java — Dynamic Routing Rule JPA Entity

- **Lines 10-36 (Fields & Constructor):** Fields: id, name, currency (EUR/USD/GBP/ALL), preferredPsp, fallbackPsp, minAmount, maxAmount, trafficWeight (0-100), and enabled boolean.
- **Lines 38-64 (Getters & Setters):** Standard accessors and mutators for dynamic rule editing via REST API.

File: routing/RoutingRuleRepository.java — Routing Rule Spring Data Repository

- **Lines 8-10:** Declares List findByEnabledTrue() for active rule filtering.

File: routing/RoutingRuleController.java — Routing Rule Management REST API

- **Lines 21-24 (GET /api/routing-rules):** Returns all configured routing rules.
- **Lines 26-33 (POST /api/routing-rules):** Creates new routing rule with generated UUID if ID is blank; returns HTTP 201.
- **Lines 35-48 (PUT /api/routing-rules/{id}):** Updates rule properties (name, currency, PSPs, thresholds, weight, enabled); returns 200 OK or 404.
- **Lines 50-57 (DELETE /api/routing-rules/{id}):** Deletes routing rule by ID; returns HTTP 204 NO CONTENT.

File: routing/PspRouter.java — Circuit-Aware Smart Routing Engine

- **Lines 30-39 (Constructor & State):** Injects List, RoutingRuleRepository, CircuitBreakerRegistry; maintains AtomicInteger cursor for round-robin balancing.
- **Lines 41-89 (selectFor Method):** 1. Evaluates matching rule via findMatchingRule. If preferred PSP is healthy (isPspHealthy(preferredId)), selects preferred. If degraded, tests fallback PSP. 2. If no rule matches or both degraded, filters healthy PSPs (isPspHealthy(id)) and round-robins. 3. If all circuits OPEN, round-robins across all to allow probe calls.
- **Lines 91-114 (findMatchingRule):** Matches active rules against transaction currency and amount range (minAmount <= amt <= maxAmount).
- **Lines 116-123 (isPspHealthy):** Checks circuit breaker state for provider; returns true if state != OPEN.

File: psp/PspClient.java — Polymorphic PSP Client Interface

- **Lines 11-18:** Declares String getId() and boolean attempt(Transaction transaction), decoupling orchestrator from concrete payment integrations.

File: psp/impl/HttpPspClient.java — Resilience-Wrapped HTTP Payment Client

- **Lines 23-39 (Constructor):** Injects id, name, endpointUrl, CircuitBreaker, simulatedFallbackFailRate, and initializes Spring RestClient with baseUrl.
- **Lines 59-70 (attempt Method):** Wraps execution in circuitBreaker.executeSupplier(() -> executePaymentCall(transaction)). Fast-fails on CallNotPermittedException when circuit is OPEN, logging warning and returning false.
- **Lines 72-112 (executePaymentCall):** Posts JSON payload (id, playerId, amount, currency, type) to mock PSP endpoint. Measures latency. Returns true if response.success == true; throws RuntimeException if declined. On connection failure, falls back to simulation to prevent demo crash.

File: psp/impl/SimulatedPspClient.java — Zero-Dependency In-Memory Mock Client

- **Lines 11-38:** Standalone mock implementation with configurable failRate; uses Math.random() to simulate approvals and declines for offline testing.

File: psp/PspController.java — PSP Monitoring & Circuit Breaker Administration

- **Lines 25-42 (PspStatusDto Record):** Exposes id, name, endpoint, masked API keys, circuitState, failureRate, avgLatencyMs, priority, supportedCurrencies, and supportedMethods.
- **Lines 45-70 (GET /api/psps):** Returns list of all registered PSPs with real-time circuit breaker states and failure rates fetched directly from CircuitBreakerRegistry.
- **Lines 72-93 (POST /api/psps/{id}/circuit-override):** Allows operations staff to manually force transition circuit breaker to OPEN, CLOSED, or HALF_OPEN.
- **Lines 95-116 (POST /api/psps/{id}/configure):** Proxies configuration updates (failureRate, avgLatencyMs, enabled) directly to mock PSP microservices on ports 8081/8082/8083.

3.5 Backend Reconciliation, KYC & Audit Domain

File: recon/ReconRun.java & ReconDiscrepancy.java — Reconciliation Data Model Entities

- **ReconRun:** Fields: id, runDate, status (COMPLETED, DISCREPANCIES_FOUND, IN_PROGRESS), totalTransactions, totalVolumeEur, matchedCount, discrepancyCount, durationSeconds, triggeredBy, createdAt.
- **ReconDiscrepancy:** Fields: id, runId, type (MISSING_IN_DB, MISSING_IN_PSP, AMOUNT_MISMATCH, STATUS_MISMATCH), reference, playerId, psp, internalAmount, pspAmount, currency, internalStatus, pspStatus, status (OPEN, RESOLVED), justificationNotes, resolvedBy, resolvedAt.

File: recon/ReconciliationService.java — Reconciliation Audit Engine & Discrepancy Resolver

- **Lines 35-41:** getRuns() and getDiscrepancies() return all stored runs and discrepancy records from database.
- **Lines 43-53 (resolveDiscrepancy):** Updates discrepancy status to RESOLVED, logs justification notes, admin resolver name, and timestamp.
- **Lines 55-178 (executeAuditRun):** Fetches all internal DB transactions; queries settlement reports from all 3 PSP mocks via GET /settlement-report (ports 8081, 8082, 8083); cross-references references; flags missing transactions in PSP or DB, amount mismatches, and status discrepancies; persists ReconRun and ReconDiscrepancy entities to database.

File: recon/ReconciliationController.java — Reconciliation REST API

- **Endpoints:** GET /api/reconciliation/runs, GET /api/reconciliation/discrepancies, POST /api/reconciliation/run (trigger audit), and POST /api/reconciliation/discrepancies/{id}/resolve (resolve discrepancy).

File: player/PlayerKYC.java & PlayerController.java — Player Compliance & KYC Limits API

- **PlayerKYC Entity:** Fields: playerId, fullName, email, country, tier (TIER_1_BASIC, TIER_2_VERIFIED, TIER_3_EDD, SUSPENDED), riskScore, documentStatus, totalDepositedEur, dailyLimitEur, dailySpentEur, lastActivity.
- **PlayerController:** GET /api/players (list all players), GET /api/players/{id} (get player KYC), and PUT /api/players/{id}/kyc (update tier, risk score, document status, or daily limit).

File: audit/AuditLog.java & AuditLogController.java — Immutable Security & Regulatory Audit Trail

- **AuditLog Entity:** Fields: id, timestamp, adminName, adminRole, action (CIRCUIT_BREAKER_OVERRIDE, ROUTING_RULE_MUTATED, KYC_TIER_UPDATE, RECON_DISCREPANCY_OVERRIDE), targetCategory, details, ipAddress.
- **AuditLogController:** GET /api/audit-logs returns complete chronological audit trail for MGA and internal compliance review.

3.6 Frontend Web Application & User Interfaces

File: frontend/lib/api-client.ts — Typed API Client & Fallback Data Layer

- **Lines 7-183 (TypeScript Type Definitions):** Declares TransactionStatus, CircuitState, Transaction, ProviderConfig, RoutingRule, ReconRun, ReconDiscrepancy, PlayerKYC, AdminUser, AdminAuditLog, AlertRule, SystemConfig, CreateTransactionRequest.
- **Lines 185-673 (Initial Seed Data):** Provides offline fallback arrays for initialProviders, initialRoutingRules, initialTransactions, initialReconRuns, initialReconDiscrepancies, initialPlayersKYC, initialAdminUsers, initialAuditLogs, initialAlertRules, and initialSystemConfig.
- **Lines 675-697 (request Helper):** Executes fetch calls with AbortController 6-second timeout; handles error throwing and JSON response deserialization.
- **Lines 699-788 (api Object):** Exports typed API methods for createTransaction, listTransactions, refundTransaction, fetchProviders, overrideCircuit, configureMockPsp, fetchRoutingRules, createRoutingRule, updateRoutingRule, deleteRoutingRule, fetchReconRuns, fetchReconDiscrepancies, triggerReconciliation, resolveDiscrepancy, fetchPlayersKYC,

updatePlayerKYC, and fetchAuditLogs.

File: frontend/app/page.tsx — Authentication & Role-Based Entry Portal

- **Lines 6-9 (Demo Accounts):** Configures demo logins: admin (ops@settleflow.dev / demo1234 -> /dashboard) and user (player@settleflow.dev / demo1234 -> /checkout).
- **Lines 11-40 (State & Handlers):** Manages email, password, error, signedIn state; fillDemo autofills credentials; handleLogin validates and redirects.
- **Lines 42-221 (Render View):** Renders centered sign-in card, input fields, demo quick-fill helper cards, and success redirect animation.

File: frontend/app/checkout/page.tsx — Player Deposit & Transaction Portal

- **Lines 23-36 (State Declarations):** Manages active tab (deposit | history), viewSettings modal, balance (€312.40), amount, payment method (Card | Bank transfer | Wallet), card number, depositState (form | processing | success | fail), and selectedTx modal.
- **Lines 37-67 (handlePay Flow):** Triggers api.createTransaction({ playerId: 'player_101', amount, currency: 'EUR', type: 'DEPOSIT' }); updates balance on SETTLED; sets success/fail UI states with fallback.
- **Lines 73-640 (Render Structure):** Renders top navigation with user avatar (AK), available balance card, deposit form with quick currency presets, instant payment method toggle, transaction history list with color-coded status badges, and transaction detail modal.

File: frontend/app/dashboard/page.tsx — Operations, Risk & Treasury Command Center

- **Lines 42-67 (Dashboard State):** Manages active tab (overview | transactions | providers), search filters, status/provider dropdowns, modal inspection, and toast notifications.
- **Lines 69-116 (fetchBackendTransactions):** Synchronously calls api.listTransactions() and api.fetchProviders(); transforms backend DTOs into dashboard state; provides toast confirmation.
- **Lines 118-150 (Filtering & Metrics):** Calculates total settled count, failure rate, retry recovery percentage, and filters transactions by search query and dropdown selections.

File: frontend/app/layout.tsx & globals.css — Root Application Layout & Design Tokens

- **layout.tsx:** Sets HTML metadata (title: 'SettleFlow - Payment Orchestration', description), viewport, and embeds globals.css.
- **globals.css:** Declares CSS custom properties (:root tokens for --bg-app, --surface-1, --surface-2, --accent, --text-primary, --text-secondary, --bg-success, --bg-danger, --bg-warning), resets, spinner keyframes, and scrollbar styling.

3.7 Mock PSP Microservices & Reconciliation Engine

File: psp-mocks/server.js — Multi-Instance Mock Payment Provider Microservices

- **Lines 10-61 (Provider Configurations):** Defines PSP Alpha (Port 8081, 5% failure rate, 120ms latency), PSP Beta (Port 8082, 60% failure rate, 750ms latency), and PSP Gamma (Port 8083, 15% failure rate, 250ms latency). Each maintains an in-memory settlementLedger.
- **Lines 63-85 (Utility Helpers):** sendJson handles CORS headers and HTTP response writing; parseJsonBody parses raw streaming request payloads.
- **Lines 87-241 (startServer Engine):** Creates HTTP servers for each PSP. Endpoints: GET /health (health check), GET /status (stats & ledger count), POST /configure (live tuning of failure rate & latency), GET /settlement-report (returns settlement ledger for reconciliation), POST /v1/payments & /v1/charges & /v2/settle (payment endpoints with latency jitter, currency validation, simulated declines, and settlement ledger logging).

File: reconciliation/reconcile.py — Standalone Settlement Reconciliation Worker

- **Lines 14-29 (fetch_json Helper):** Queries JSON endpoints with timeout protection using urllib.request.
- **Lines 30-64 (Data Gathering):** Fetches internal transactions from BACKEND_URL/api/transactions; fetches settlement ledgers from all 3 PSPs via /settlement-report.
- **Lines 65-131 (Audit Logic):** Compares internal DB records with PSP records. Identifies MISSING_IN_PSP (settled in DB but absent from PSP ledger), AMOUNT_MISMATCH (differing amounts), STATUS_MISMATCH (status discrepancies), and MISSING_IN_DB (settled at PSP but missing internally).
- **Lines 133-159 (Report Generator):** Prints formatted ASCII audit table and exports full audit details to reconciliation_report.json.

3.8 Kubernetes Manifests & CI/CD Pipelines

File: k8s/namespace.yaml & configmap.yaml — Cluster Namespace & Environment Configuration

- **namespace.yaml:** Defines isolated settleflow Kubernetes namespace.
- **configmap.yaml:** Stores environment variables: SPRING_DATASOURCE_URL, CORS_ALLOWED_ORIGIN, PSP_ALPHA_URL, PSP_BETA_URL, PSP_GAMMA_URL, and NEXT_PUBLIC_BACKEND_URL.

File: k8s/postgres.yaml & psp-mocks.yaml — Database & Mock PSP Deployments

- **postgres.yaml:** Deploys postgres:16-alpine with 1Gi PersistentVolumeClaim and ClusterIP service on port 5432.
- **psp-mocks.yaml:** Deploys settleflow-psp-mocks exposing container ports 8081, 8082, and 8083 via psp-mocks-service.

File: k8s/backend.yaml & frontend.yaml — Application Core Deployments & Services

- **backend.yaml:** Deploys settleflow-backend with liveness and readiness HTTP probes on /api/transactions, memory limits (512Mi), and backend-service ClusterIP on port 8080.
- **frontend.yaml:** Deploys settleflow-frontend with frontend-service ClusterIP on port 3000.

File: k8s/reconciliation-cronjob.yaml — Scheduled Reconciliation CronJob

- **Spec & Schedule:** Runs schedule: '0 6 * * *' (daily at 06:00 UTC); executes settleflow-recon container image with BACKEND_URL=http://backend-service:8080.

File: k8s/ingress.yaml & public-tunnel.yaml — Ingress Controller & Cloudflare Public Tunnel

- **ingress.yaml:** Nginx Ingress routing / to frontend-service:3000 and /api to backend-service:8080.
- **public-tunnel.yaml:** Deploys cloudflare/cloudflared sidecar connecting frontend-service:3000 to public zero-trust URL.

File: .github/workflows/ci.yml & cd.yml — Automated GitHub Actions Pipelines

- **ci.yml Jobs:** 1. backend-build (Java 17 Temurin, mvn clean test), 2. frontend-build (Node 20, npm ci, tsc --noEmit, eslint, next build), 3. psp-mocks-check (health curl tests), 4. reconciliation-check (Python 3.11 test run).
- **cd.yml Jobs:** Builds multi-arch Docker container images and pushes to GitHub Container Registry (ghcr.io).

4. Production Security, High-Availability & Compliance

Operating financial transaction infrastructure requires strict adherence to international regulatory standards, data integrity guarantees, and zero-downtime architecture.

- **Exact Financial Precision (BigDecimal / NUMERIC):** Floating-point IEEE-754 numbers (float/double) suffer from binary rounding errors (e.g. 0.1 + 0.2 = 0.30000000000000004). SettleFlow strictly enforces Java BigDecimal and PostgreSQL

NUMERIC(19,2) across all entities and calculations, guaranteeing zero loss of currency cents.

- **Idempotency Keys & Safe Retries:** Network drops between merchants and payment gateways often cause retried requests. SettleFlow intercepts incoming requests via unique idempotency keys in TransactionService.create(). If a key was already processed, the original transaction is returned immediately without re-triggering upstream payment charges.
- **PCI-DSS Level 1 Data Isolation:** Primary Account Numbers (PANs), CVVs, and raw card details are never persisted in the SettleFlow database. All sensitive tokens, API secrets, and webhook signatures are masked in API responses (e.g. ak_live_••••••••90a1).
- **MGA / AML Regulatory Compliance:** Player KYC tiers (Tier 1 Basic through Tier 3 Enhanced Due Diligence) enforce automated velocity checks and daily deposit limits. Every administrative action (circuit overrides, KYC adjustments, manual reconciliations) generates an immutable AuditLog record capturing admin name, timestamp, action, and client IP.
- **Graceful Degradation & Self-Healing:** If all external PSP microservices become unavailable, the frontend and backend fall back to simulated zero-downtime execution modes, preventing user checkout crashes during localized network outages.

5. Future Enhancements & Strategic Roadmap

To scale SettleFlow to handle tens of thousands of transactions per second across global tier-1 merchants, the following architectural upgrades are planned for subsequent releases:

Event-Driven Architecture with Apache Kafka	Kafka Event Bus & Outbox Pattern	Decouple synchronous HTTP request handling. Inbound transactions are written to Kafka topics (payment.requested, payment.settled), enabling asynchronous worker pools to process 50,000+ tx/sec without blocking HTTP threads.
Distributed Locking with Redis / Redlock	Redis Distributed Lock Cluster	Enforces cluster-wide single-execution locks on player accounts and idempotency keys across multi-replica Spring Boot backend pods, preventing concurrent double-spend race conditions.
Multi-Region Active-Active Sharding	CockroachDB / AWS Aurora Global	Enables multi-region transaction routing with sub-50ms latency for European, North American, and Asian players while maintaining strict serializable consistency.
ML-Powered Dynamic Routing & Fraud Engine	Real-Time Python ML Sidecar	Analyzes real-time provider acceptance rates, card issuer bins, interchange fees, and player fraud risk scores to dynamically pick the cheapest PSP with the highest likelihood of approval.
Open Banking & Instant Rail Adapters	ISO 20022 / SEPA Instant / FedNow	Native connectors for instant bank-to-bank account settlements (SEPA Instant, UK Faster Payments, FedNow, Pix, UPI), bypassing traditional card interchange fees.
Automated Reconciliation Auto-Healer	Self-Healing Workflow Engine	Automatically resolves simple discrepancies (e.g. status lag where PSP settled 30s after backend timeout) via automated secondary status polling and balance adjustments.

6. Complete REST API Reference Directory

Complete endpoint catalog exposed by the SettleFlow Spring Boot transaction orchestrator on port 8080:

HTTP Method & Path	Request Body / Params	Response	Description
POST /api/transactions	CreateTransactionRequest	201 Created (Transaction)	Creates, deduplicates, routes, and processes a deposit/withdrawal.

GET /api/transactions	*None*	200 OK (List<Transaction>)	Retrieves all transaction records.
GET /api/transactions/{id}	UUID id (Path)	200 OK / 404 Not Found	Retrieves single transaction by ID.
POST /api/transactions/{id}/refund	{ amount?, reason? }	200 OK (Transaction)	Refunds a settled transaction.
GET /api/psps	*None*	200 OK (List<PspStatusDto>)	Returns all PSPs with live circuit breaker states.
POST /api/psps/{id}/circuit-override	{ targetState: 'OPEN'|'CLOSED' }	200 OK	Manually forces circuit breaker state.
POST /api/psps/{id}/configure	{ failureRate?, avgLatencyMs? }	200 OK	Tunes mock PSP microservice parameters live.
GET /api/routing-rules	*None*	200 OK (List<RoutingRule>)	Lists all active and inactive routing rules.
POST /api/routing-rules	RoutingRule	201 Created	Creates a new routing rule.
PUT /api/routing-rules/{id}	RoutingRule	200 OK	Updates an existing routing rule.
DELETE /api/routing-rules/{id}	String id (Path)	204 No Content	Deletes a routing rule.
GET /api/reconciliation/runs	*None*	200 OK (List<ReconRun>)	Lists historical audit runs.
GET /api/reconciliation/discrepancies	*None*	200 OK (List<Discrepancy>)	Lists all reconciliation discrepancies.
POST /api/reconciliation/run	{ triggeredBy? }	200 OK (ReconRun)	Triggers immediate settlement ledger audit.
POST /api/reconciliation/discrepancies/{id}/resolve	{ justification?, adminName? }	200 OK	Marks discrepancy as resolved with audit note.
GET /api/players	*None*	200 OK (List<PlayerKYC>)	Lists all player KYC profiles.
PUT /api/players/{id}/kyc	{ tier?, riskScore?, dailyLimit? }	200 OK (PlayerKYC)	Updates player compliance tier and limits.
GET /api/audit-logs	*None*	200 OK (List<AuditLog>)	Returns chronological compliance audit trail.

Handbook Conclusion & Verification: This document serves as the single source of truth for the SettleFlow architecture. All components have been verified for compilation, containerization, local execution, and Kubernetes deployment. For live testing, refer to the quickstart commands in NOTES.md or run deploy.ps1 -Compose.